

Modelos De Custo

João Eduardo Montandon
DCC024

Vamos relembrar a implementação do método append.

```
append([], L, L).
append([H|T], L, [H|Tt]) :- append(T, L, Tt).
```

```
?- length(L, 5000000), time(append([a], L, LL)).
?- length(L, 5000000), time(append(L, [a], LL)).
```

Qual consulta é mais eficiente? 🤔

Uma mesma tarefa pode ser implementada de diversas formas. Qual é a **melhor**?

Modelo de custo é um mapa mental dos custos das operações presentes em uma linguagem de programação.

- Não está descrito na especificação da linguagem.
- Uma percepção que se adquire com o tempo e contato com a linguagem.

Modelos De Custo

1. Listas
2. Chamadas de Função
3. Busca em Prolog
4. Arrays
5. Outros Custos

Listas

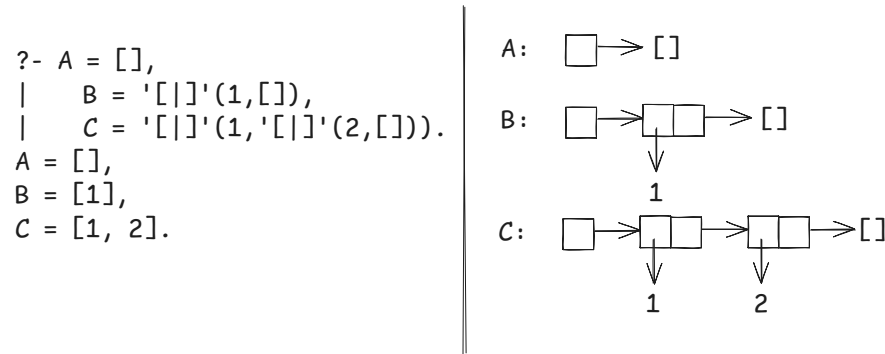
Como listas são implementadas em Prolog ou SML? 🤔

```
val cons = op ::;
val L = cons(1,cons(2,cons(3,[])));
> val it = [1, 2, 3];
```

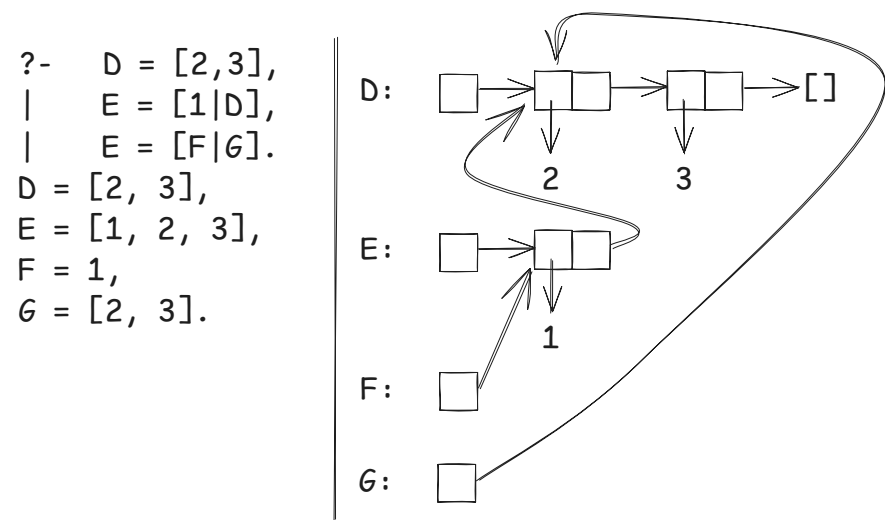
```
?- L = '[]'(1,'[]'(2,'[]'(3,[]))).
L = [1, 2, 3].
```

- São implementas como uma lista encadeada simples (**Cons Cell**):
(valor, *proximo)
- Qual o custo para inserir na cabeça da lista? 🤔
- Qual o custo para inserir no fim da lista? 🤔

Operador Cons



Vantagem: Compartilhamento de Listas



Como sabemos que as listas são compartilhadas? 🤔

- Isso é detalhe de implementação da linguagem.
- Só experimentando para saber.
- `E = [1|D]` é resolvida em uma quantidade constante de tempo e espaço.

Tamanho Da Lista

```
fun length nil = 0  
| length (head::tail) = 1 + length tail;
```

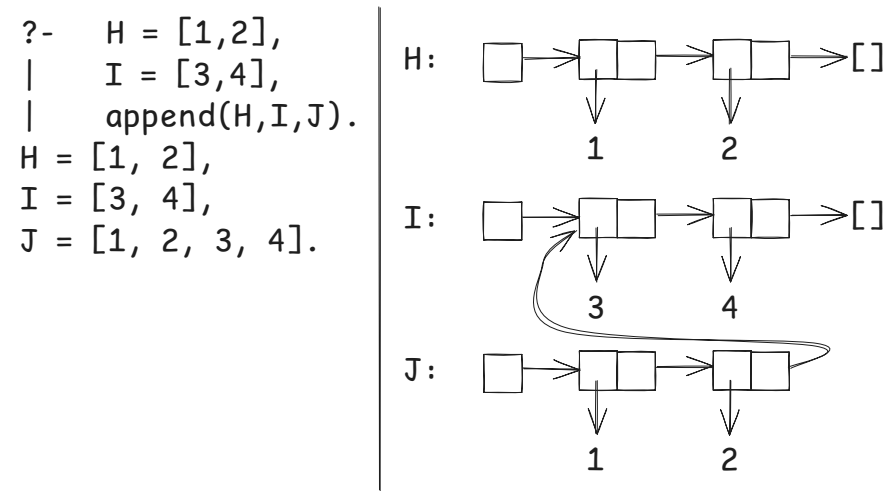
Qual o custo? 🤔

Função append

Qual seria a representação gráfica das listas abaixo?

```
?- H = [1,2],
|   I = [3,4],
|   append(H,I,J).
H = [1, 2],
I = [3, 4],
J = [1, 2, 3, 4].
```

Alguma estrutura será compartilhada? Qual? 🤔

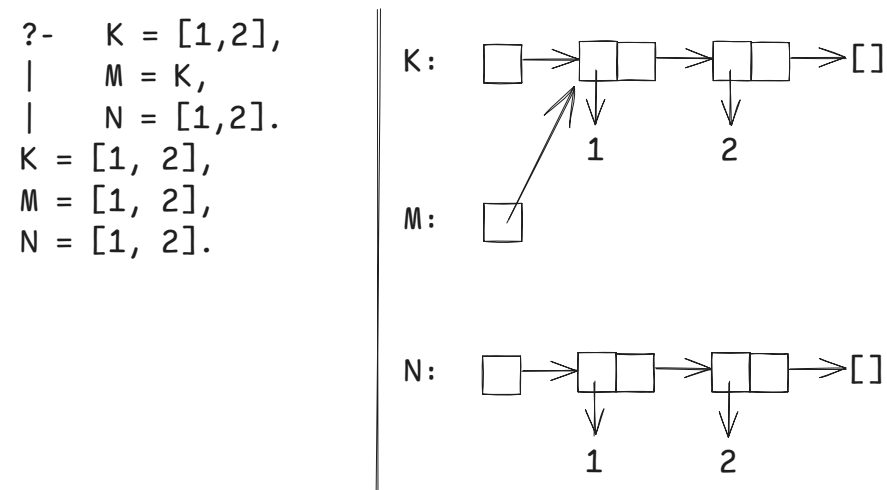


Qual o custo? 🤔

Unificação

```
?- K = [1,2],
|   M = K,
|   N = [1,2].
K = [1, 2],
M = [1, 2],
N = [1, 2].
```

- Qual o custo para verificar `K = M`?
- Qual o custo para verificar `K = N`? 🤔



Sumarizando...

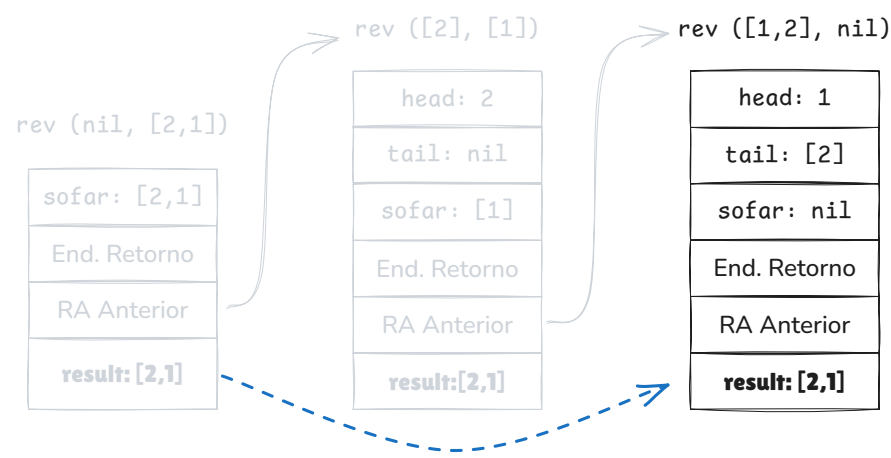
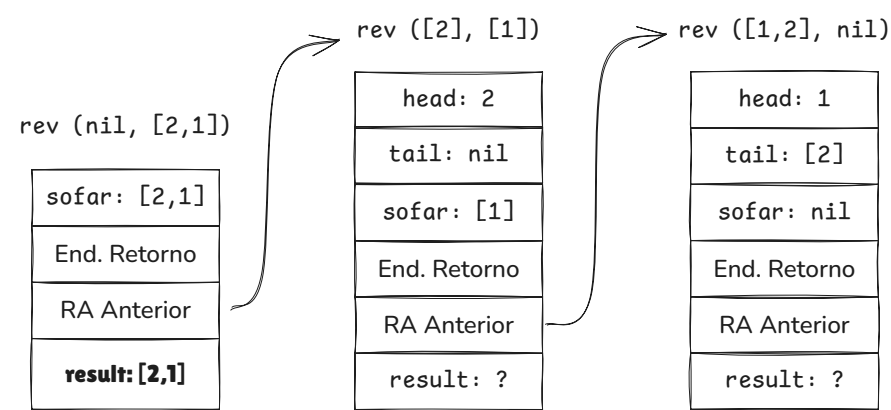
- Custo de *Cons* é constante.
- Extrair a cabeça ou cauda é constante.

- Custo de `length` é proporcional a lista.
- Custo do `append` é proporcional a primeira lista.
- Custo de comparar duas listas é, no pior caso, proporcional ao tamanho de ambas.

Chamadas De Função

```
fun reverse x =  
  let  
    fun rev(nil, acc) = acc  
    |   rev(head::tail, acc) = rev(tail, head::acc);  
  in  
    rev(x, nil)  
  end;
```

Desenhe os Registros de Ativação (RA) para `reverse [1,2]`.



O que aconteceu com `result` ?

Recursividade De Cauda

Uma **chamada de cauda** acontece quando a chamada da função apenas retorna um resultado para seu *caller*.

A **recursividade de cauda** acontece quando todas as chamadas são de cauda.

Quais otimizações são possíveis? 🤔

- Suas variáveis e parâmetros não são necessários, pois não há computação pendente.
- Neste caso, o RA pode ser reaproveitado pela próxima chamada.

- Na prática, um único RA gerencia toda a recursividade.

Qual o custo de espaço? Que impacto essa otimização traz para a robustez do programa? 🤔

Exemplo

Recursividade de cauda? 🤔

```
fun length nil = 0
|   length (h::t) = 1 + length t;
```

Agora sim! 😎

```
fun length l =
  let
    fun len (nil,acc) = sofar
    |   len (h::t,acc) = len (t,acc+1);
  in
    len l
  end;
```

Observações Importantes

- `acc` ➡ utilizado para "carregar" a computação entre as chamadas.
- Implementado em praticamente todo sistema de linguagem.
- Especialmente importante em linguagens funcionais.

Mas E Prolog?

```
p(X) :- q(X), p(X-1).
```

`p` é uma chamada de cauda? 🤔

Depende! Devido ao *backtracking*, nem sempre a cláusula ao fim do predicado é a última a ser executada.

Se `p(X-1)` falhar, o backtracking pode continuar a prova a partir de `q(X)`.

Buscas Em Prolog

Considere o seguinte programa em Prolog:

```
parent(d, a).
parent(f, e).
parent(a, b).
```

```
parent(e, b).
parent(e, c).
male(d).
male(a).
```

Qual regra é mais eficiente? 🤔

```
grandfather(X,Y) :- parent(X,Z), parent(Z,Y), male(X).
```

```
grandfather(X,Y) :- male(X), parent(X,Z), parent(Z,Y).
```

- Recomendação: declare cláusulas mais restritivas primeiro.
- Assim estaremos "podando" a árvore de busca.

Arrays

```
int sumByRow(int m[1000][1000]) {
    int total = 0;
    for(int i = 0; i < 1000; i++)
        for(int j = 0; j < 1000; j++)
            total += m[i][j];
    return total;
}
```

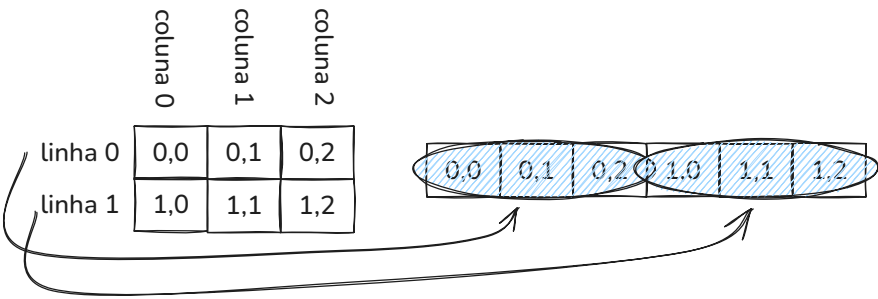
```
int sumByColumn(int m[1000][1000]) {
    int total = 0;
    for(int j = 0; j < 1000; j++)
        for(int i = 0; i < 1000; i++)
            total += m[i][j];
    return total;
}
```

Qual implementação é mais eficiente?

Depende!

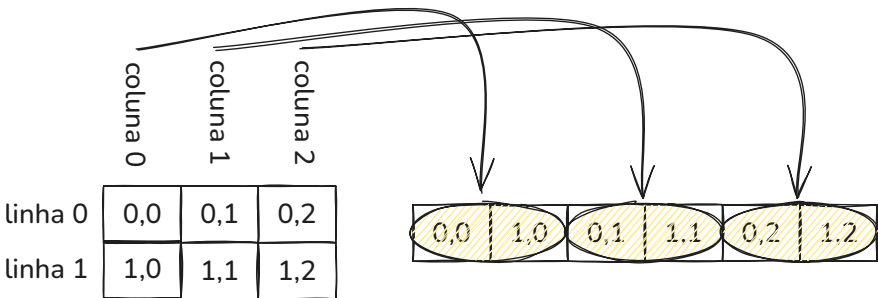
- Hardware é otimizado para acesso sequencial de dados.
- Todo array é reservado em um espaço contíguo da memória.
- Em qual ordem seus dados são armazenados? 🤔

Row-Major



Exemplo: C/C++

Column-Major



Exemplo: Fortran

Como Saber?

- Depende do que a linguagem lhe expõe.
- C/C++: aritmética de ponteiros.

```
int main() {
    int m2[2][3];
    int *p = &m2[0][0];

    printf("m2[0][0] = p? %d\n", &m2[0][0] == p);
    printf("m2[0][1] = p+1? %d\n", &m2[0][1] == p+1);
    printf("m2[1][0] = p+1? %d\n", &m2[1][0] == p+1);

    return 0;
}
```

O que vai ser impresso nesse programa? 🤖

Considerações Finais

- Conhecer como uma linguagem implementa seus recursos permite criar programas mais eficientes.
- Nem todos os custos estão na especificação da linguagem.
- Conhecer os modelos de custos são importante, mas nada substitui bons algoritmos!